Exercises SELECT 1 CLIENT (SEQUENCE,SYNONYM,DATE)

Exercises SELECT-1 (Customer's tables)

```
-- 1 (No me funciona de forma manual)
INSERT INTO CO_CUSTOMER
(DNI, NAME, PROVINCE_CODE, TYPE, REGISTER_DATE, VENDOR_CODE, TOTAL_PURCHASES)
VALUES
    (:dni, :name, :province_code, :type, TO_DATE(:register_date, 'DD/MM/RR'), :vendor_code, :total_purchases);
commit;
-- 2 The name of each customer with their purchases (in thousands)
select NAME,round(TOTAL_PURCHASES,-3) as "Rounded in thousands" from CO_CUSTOMER;

-- 3  List of customers sorted by customer type and name (in descending order)
select NAME,TYPE from CO_CUSTOMER order by TYPE desc;

-- 4 List of customers of type 'AA' to be in Zaragoza and Barcelona
select NAME from CO_CUSTOMER where TYPE='AA' and PROVINCE_CODE in ('BA','ZA');

-- 5. Name of the provinces where there are customers
select PROVINCE_CODE from CO_CUSTOMER group by PROVINCE_CODE;

-- 6.  Total Zaragoza's customer purchases
select sum(TOTAL_PURCHASES) from CO_CUSTOMER where PROVINCE_CODE='ZA';

-- 7. Minimum, Maximum, Average from the purchases of our clients
select min(TOTAL_PURCHASES),max(TOTAL_PURCHASES),avg(TOTAL_PURCHASES) from CO_CUSTOMER;

-- 8. How many customers and how many different types of customers are in the province of Zaragoza
select count(*),count(distinct TYPE) from CO_CUSTOMER where PROVINCE_CODE='ZA';

-- 9. How many days has been each customer in our table (the data should not have decimal)
select count(distinct TO_CHAR(REGISTER_DATE,'DD')) from CO_CUSTOMER;

-- 10. Sum of purchases by province
select PROVINCE_CODE,sum(TOTAL_PURCHASES) from CO_CUSTOMER group by PROVINCE_CODE;

-- 11. How many clients of each type is in each province. The list is sorted by the total amount of
-- purchases from each group
```

```
select TYPE,PROVINCE_CODE,count(*) from CO_CUSTOMER group by TYPE,PROVINCE_CODE order by count(*) desc;

-- 12. Report how many days has the oldest client in each provinces.
select PROVINCE_CODE,min(TO_CHAR(REGISTER_DATE,'DD')) from CO_CUSTOMER group by PROVINCE_CODE;

-- 13. How many clients are in each group and what is the total amount of his/her purchases, as long as this
-- amount exceeds 150000.
select PROVINCE_CODE,count(*),sum(TOTAL_PURCHASES) from CO_CUSTOMER
where TOTAL_PURCHASES>150000 group by PROVINCE_CODE;

-- 14.  Erase the customers of the province 'MA'
delete from CO_CUSTOMER where PROVINCE_CODE='MA';
commit;

-- 15. Increase 10% of the value of customers' purchases of type 'AA'
update CO_CUSTOMER set TOTAL_PURCHASES=TOTAL_PURCHASES*1.1 where TYPE='AA';
commit;

-- 16. create a sequence to insert the table five article items with codes: 1010, 1020,
-- 1030, 1040 and 1050.
create sequence seq_item_code start with 1010 increment by 1 nocycle ;
commit;
insert into CO_ITEM (CODE,DESCRIPTION,PRICE,STOCK) values (seq_item_code.nextval,'item1',10,100);
commit;
insert into CO_ITEM (CODE,DESCRIPTION,PRICE,STOCK) values (seq_item_code.nextval,'item2',20,200);

create sequence seq_item_code start with 1030 increment by 10 nocycle;
insert into CO_ITEM (CODE,DESCRIPTION,PRICE,STOCK) values (seq_item_code.nextval,'item3',30,300);
insert into CO_ITEM (CODE,DESCRIPTION,PRICE,STOCK) values (seq_item_code.nextval,'item4',40,400);
insert into CO_ITEM (CODE,DESCRIPTION,PRICE,STOCK) values (seq_item_code.nextval,'item5',50,500);
```

Exercises CREATE / SYNONYM / DATE (Table staff and department)

```
-- 1
CREATE TABLE STAFF (
   -- Cod_emp: Primary key
                 Cod_emp varchar2(4) PRIMARY KEY,

   -- Name: Different value for each queue (UNIQUE)
```

```sql
                Name VARCHAR(100) UNIQUE,

    -- Job: Only allows specific values (CHECK)
                Job VARCHAR(20) CHECK (Job IN ('director', 'analyst', 'programmer', 'project manager')),

    -- Salary: Values between 3000 and 10000 (CHECK)
                Salary DECIMAL(10, 2) CHECK (Salary BETWEEN 3000 AND 10000),

    -- Date of entry: The default value is the system date (DEFAULT).
                Date_entry DATE DEFAULT CURRENT_DATE,

    -- Cod_dept: Foreign key, cannot be null (NOT NULL)
                Cod_dept varchar(10) NOT NULL,

    -- Cod_sup: Default value is zero (DEFAULT)
                Cod_sup INT DEFAULT 0,

    -- Definition of the foreign key
                FOREIGN KEY (Cod_dept) REFERENCES DEPARTMENT(DEPARTMENT_CODE)
);
create table DEPARTMENT
(
    DEPARTMENT_CODE VARCHAR2(10) not null
        primary key,
    NAME            VARCHAR2(50),
    CITY            VARCHAR2(50)
);
-- 2

INSERT INTO STAFF (Cod_emp, Name, Job, Salary, Cod_dept, Cod_sup)
VALUES ('0368', 'Acebo', 'project manager', 9700, '12', NULL);
INSERT INTO STAFF (Cod_emp, Name, Job, Salary, Cod_dept, Cod_sup)
VALUES ('1008', 'Garcia', 'project manager', 7800, '10', '0368');
INSERT INTO STAFF (Cod_emp, Name, Job, Salary, Cod_dept, Cod_sup)
VALUES ('1190', 'Ghegoiu', 'project manager', 8000, '10', '0368');
INSERT INTO STAFF (Cod_emp, Name, Job, Salary, Cod_dept, Cod_sup)
VALUES ('0413', 'Bennasar', 'analyst', 6000, '05', '1008');
INSERT INTO STAFF (Cod_emp, Name, Job, Salary, Cod_dept, Cod_sup)
VALUES ('0545', 'Cerda', 'analyst', 5600, '05', '1008');
INSERT INTO STAFF (Cod_emp, Name, Job, Salary, Cod_dept, Cod_sup)
VALUES ('0552', 'De Dios', 'analyst', 5500, '05', '1190');
INSERT INTO STAFF (Cod_emp, Name, Job, Salary, Cod_dept, Cod_sup)
VALUES ('0663', 'De Prada', 'analyst', 6700, '05', '1190');
INSERT INTO STAFF (Cod_emp, Name, Job, Salary, Cod_dept, Cod_sup)
VALUES ('1003', 'Franko', 'analyst', 6600, '05', '1190');
INSERT INTO STAFF (Cod_emp, Name, Job, Salary, Cod_dept, Cod_sup)
VALUES ('0765', 'Deya', 'programmer', 3800, '08', '0413');
INSERT INTO STAFF (Cod_emp, Name, Job, Salary, Cod_dept, Cod_sup)
VALUES ('0998', 'Fedeli', 'programmer', 4300, '08', '0413');
INSERT INTO STAFF (Cod_emp, Name, Job, Salary, Cod_dept, Cod_sup)
```

```sql
VALUES ('1087', 'Gelabert', 'programmer', 4000, '08', '1003');

insert into STAFF (Cod_emp, Name, Job, Salary, Cod_dept, Cod_sup)
values ('0998', 'Petraroia', 'analyst', 5700, '05', '0444');
-- It fails because the PK is repeated.

insert into  STAFF (Cod_emp, Name, Job, Salary, Cod_dept, Cod_sup)
values ('1004', 'Rigo', 'driver', 7500, '10', NULL);
--It fails because the job cannot be a “driver.”

insert into STAFF (Cod_emp, Name, Job, Salary, Cod_dept, Cod_sup)
VALUES ('1200', 'Rodriguez', 'project manager', 2500, '10', '0413');
-- The salary of 2500 is less than the minimum of 3000

insert into STAFF (Cod_emp, Name, Job, Salary, Cod_dept, Cod_sup)
VALUES ('1110', 'Sanchez', 'programmer', 4000, '06', '1008');
-- The department code 06 does not exist.
```

The use of constraints is not only a matter of order, but also of security and efficiency:

- **Data Integrity**: Ensures that the information in the database is logical and consistent (for example, it does not allow negative salaries or non-existent positions).
- **Prevention of Human Error**: Acts as an automatic filter that prevents a user or application from accidentally entering incorrect data.
- **Referential Consistency**: Prevents "orphan data" (such as an employee assigned to a department that does not exist).
- **Code Simplification**: There is no need to write complex validations in the programming language (Java, Python, etc.), as the database itself rejects anything that does not comply with the rules.

```sql
-- 3 Modify data so they can be accepted for the database.
-- 3.1. Petraroia: The code is changed from 0998 to 1301 because 0998 already exists.
INSERT INTO STAFF (Cod_emp, Name, Job, Salary, Cod_dept, Cod_sup)
VALUES (1301, 'Petraroia', 'analyst', 5700, 05, 0413);

-- 3.2. Rigo: The position is changed from 'driver' to 'programmer' (permitted value).
INSERT INTO STAFF (Cod_emp, Name, Job, Salary, Cod_dept)
VALUES (1004, 'Rigo', 'programmer', 7500, 10);

-- 3.3. Rodriguez: The code is changed to 1302 and the salary is raised to the minimum of 3000.
INSERT INTO STAFF (Cod_emp, Name, Job, Salary, Cod_dept, Cod_sup)
VALUES (1302, 'Rodriguez', 'project manager', 3000, 10, 0413);
```

```sql

-- 3.4. Sanchez: Department 06 is changed to 05 because 06 does not exist in DEPARTMENT.
INSERT INTO STAFF (Cod_emp, Name, Job, Salary, Cod_dept, Cod_sup)
VALUES (1110, 'Sanchez', 'programmer', 4000, 05, 1008);

```

4.

1. In the DEPARTMENT table: Add the UNIQUE constraint.

For a column to be referenced by another table as a foreign key, that column **must be unique** (or be the primary key). Currently, DEPARTMENT_CODE is the primary key, but if we want to reference NAME, we must ensure its uniqueness.

- **Constraint to add:** UNIQUE on the Name column.
- **Reason:** Without this constraint, the database would not allow STAFF.Job to point to DEPARTMENT.Name, as there would be no guarantee that each department name is unique.

2. In the STAFF table: Modify the CHECK constraint to a FOREIGN KEY

The CHECK (Job IN (...)) constraint that we initially defined is static and only allows values that are manually written in the code (director, analyst, etc.).

- **Constraint to modify:** Remove the CHECK and replace it with a **Foreign Key (FOREIGN KEY)** that points to DEPARTMENT(Name).
- **Reason:** This dynamically links both tables. If a new department called "Research" is created in the future, "Research" automatically becomes a valid job position in the STAFF table without having to alter the table structure.

5.

Command	Expected Result	Explanation
`SELECT sysdate FROM dual;`	`11-FEB-26`	Returns the current system date in the database's default format.
`SELECT to_char(sysdate, 'DD-MM-YYYY') FROM dual;`	`11-02-2026`	Formats the date using numeric day, month, and 4-digit year.
`SELECT to_char(sysdate, 'YYDDD') FROM dual;`	`26042`	`YY` is the last two digits of the year (26); `DDD` is the day of the year (042).
`SELECT to_char(sysdate, 'Q') FROM dual;`	`1`	Returns the current **Quarter** of the year (1st Quarter).

6.

1. Select sysdate from dual;

- **Result:** Displays the current system date and time in the database's default format (usually DD-MON-YY).
- **Explanation:** sysdate is a function that returns the current server date. dual is a special single-row, single-column table used in Oracle to evaluate expressions that do not require data from an actual table.

2. Select to_char (sysdate, 'DD-MM-YYYY') from dual;

- **Result:** 11-02-2026 (based on today's date).
- **Mask:**
- DD: Day of the month in numerical format (01-31).
- MM: Month in numerical format (01-12).
    - YYYY: Full year with 4 digits.

3. Select to_char (sysdate, 'YYDDD') from dual;

- **Result:** 26042.
- **Mask:**
    - YY: The last two digits of the year (26).

- DDD: The day of the year (Julian calendar), ranging from 001 to 366. 11 February is day 042 of the year.

4. Select to_char(sysdate, 'Q') from dual;

- **Result:** 1.
- **Mask:**
  - Q: Returns the **quarter** of the year (1, 2, 3 or 4). As it is February, the result is the first quarter.

5. Complex query on the STAFF table

Select to_char (Date_entry, '"Joined in " Ddsp " of " fmMonth " of " YYYY " at " HH:MI " am"') from staff;

- **Result:** "Joined in Eleventh of February of 2026 at 01:48 am" (for example).
- **Explanation of the mask:**
- **"Text"**: Any text between double quotation marks is printed literally.
- **Ddsp**: The day of the month written in English words (Eleven) with the suffix sp (*spelled*) spelling it out. The first capital D makes the first letter uppercase.
- **fmMonth**: The full name of the month. The prefix fm (*fill mode*) removes the extra white space that Oracle usually adds to fill.
- **YYYY**: 4-digit year.
  - **HH:MI**: Time (12-hour format) and minutes.
  - **am**: Displays the morning or afternoon indicator.

7.

```
-- Updating the entry date for employee 1008 (Garcia)
UPDATE STAFF
SET Date_entry = TO_DATE('2023-05-15 14:30', 'YYYY-MM-DD HH24:MI')
WHERE Cod_emp = 1008;

-- Updating the entry date for employee 1190 (Ghegoiu)
UPDATE STAFF
```

```
8   SET Date_entry = TO_DATE('2022-09-01 09:15', 'YYYY-MM-DD HH24:MI')
9   WHERE Cod_emp = 1190;
```